

Technical Decision Record: Digital Direct Channel Strategy

Metadata

Field	Detail
Status	Accepted
Date	May 2026
Decider	Andres Velandia
Context	Multi-country Insurtech Platform — Digital Direct Channel v1.0

1. Context

We are deploying a multi-country Digital Direct Channel that must support mission-critical business flows — Policy Issuance, Payments, and Quotes — operating simultaneously across multiple regions with distributed teams. We must define a governance model and communication patterns that ensure high availability, scalability, and rapid time-to-market without creating technical bottlenecks.

2. Integration Governance: Federated vs. Centralized

	Option A: Centralized (<i>Rejected</i>)	Option B: Federated with Center for Enablement (<i>Selected</i>)
Description	One central team manages all integrations.	Regional teams own their integrations, supported by a central enablement team.
Key reason	Creates a structural bottleneck; scales poorly as countries are added and concentrates knowledge in a single point of failure.	Maximizes regional autonomy and delivery speed; the enablement team ensures global consistency through standards and reusable assets.

Decision: Regional teams must own their delivery. The enablement team provides common foundations — API contracts, security standards, and shared libraries — while each region executes autonomously.

3. Communication Patterns: Event-Driven vs. Synchronous

	Option A: Synchronous (<i>Limited use</i>)	Option B: Event-Driven Architecture (<i>Selected</i>)
Description	Real-time request-response communication.	Asynchronous messaging with guaranteed delivery via an event broker.
Key reason	Introduces temporal coupling; a receiver failure at the exact moment of the request results in a lost transaction. Suitable only for read-only operations.	Decouples the frontend from the core; guarantees processing even under load through queues with automatic retries and dead letter queues.

Decision: For mission-critical flows such as Policy Issuance and Payments, EDA ensures transactions are reliably queued and processed, preventing data loss and delivering a resilient user experience.

4. Consequences & Risks

Risk	Mitigation
Operational complexity — Asynchronous flows are harder to trace and debug.	Full-stack observability with Correlation ID propagated across all layers.
Eventual consistency — The frontend must handle intermediate states ("processing", "pending").	Explicit state contracts in the experience-layer APIs; the frontend uses polling or webhooks to reflect the final state.
Federated model learning curve — Regional teams must internalize standards before executing autonomously.	Initialize the enablement team early, with documentation and active support during the first weeks of adoption.